Assignment 1

Scheduling

AUTHOR
Operating Systems Teaching and Research Unit

PUBLISHED
14.05.2024

General information

This Tuesday will be your first graded assignment. You have from May, 14th to May, 28th at 4PM to complete it.

You must submit before the deadline. No extension of deadline will be given.

You are encouraged to evaluate your code gradually using the evaluation features of VPL.

> **Tip**
>
> Some IDE options can be changed on VPL, they can be displayed by pressing CTRL+,

Assignment

This week's assignment will focus on implementing two scheduling algorithms presented during the lecture:

- **FCFS**
- **Round Robin**

In this assignment, every process will be represented by a structure `task`, with the following fields :

- a `pid`, for process ID
- a `state`, which can be either `BLOCKED`, `RUNNING`, `READY` or `TERMINATED`

```
struct task {
    int pid;
    enum states state;
    struct task* prev;
    struct task* next;
};
```

In addition to these fields, we add two pointers to the next and previous tasks. It allows us to link tasks together in a linked list.

This is how the run queue will be implemented, using a doubly linked list :

```c
struct run_queue {
    // head of the linked list
    struct task* head;
    // number of task in the run queue
    size_t n_tasks;
    // timeslice left for the running task
    int time_counter;
};
```

Task 1: Linked Lists

Start by implementing the functions related to the linked lists inside `doubly_linked_list.c`.

Make sure that your implementation is correct by evaluating your code before proceeding to the next task.

> **Tip**
>
> If you are unable to complete the first task, you can use our own run queue implementation in task 2 & 3.
>
> Include the header and call the functions:
>
> ```c
> #include "doubly_linked_list_solution.h"
>
> //remove the "stud_" prefix and call the functions
> rq_empty(rq);
> task_free(task);
> ```
>
> But don't try to use them for task 1, you won't get the points ;)

Task 2: FCFS

For each scheduling event, a function is called and will modify the run queue and/or modify the state of the tasks.

```c
enum events {
    start,
    wait,
    wake_up,
    terminate,
    clock_tick,
};
```

Using the functions implemented in task 1, now implement the different functions for **FCFS**. Remember that **FCFS** is a non-preemptive scheduling algorithm.

- `stud_FCFS_start`
- `stud_FCFS_elect`
- `stud_FCFS_terminate`
- `stud_FCFS_clock_tick`

- `stud_FCFS_wait`
- `stud_FCFS_wake_up`

Each function is described in the C file with a comment. Complete these functions in `scheduler_fcfs.c`.

Task 3: Round Robin

Similar to task 2, implement the Round Robin functions in `scheduler_round_robin.c`.